

SYSTEM DESIGN DEEP-DIVE

Feature Flags at Scale

How Google, Meta, Netflix & Uber use feature flags as a
distributed control plane – not config files

\$440M

ONE UN-DELETED FLAG

<1ms

LOCAL EVAL LATENCY

<30s

GLOBAL KILL SWITCH

THE MOST EXPENSIVE DEPRECATED FLAG IN HISTORY.

\$440M

LOST IN 45 MINUTES FROM ONE UN-DELETED FEATURE FLAG

Knight Capital. 2012.

This Is Not a Config File Problem

At scale, feature flags become a distributed control system.

- Config files break under concurrent deploys across 1,000 servers
- You need a control plane – not a YAML file
- Two planes: Control (slow/safe) and Data (fast/local)
- Data plane runs entirely in-process – no remote calls on hot path
- Rule one: user traffic must NEVER block on a remote flag service
- Google, Meta, Netflix, Uber all converged on this independently

The Kill Switch Comes First

Evaluated before targeting rules. No user context required.

- Kill switches are boolean overrides – on or off, globally
- They run before percentage rollouts and user targeting
- No userId needed – evaluate instantly from local cache
- Uber disabled surge pricing globally in under 30 seconds
- This is the circuit breaker for your product behavior
- Design kill switches into every flag from day one

GLOBAL KILL SWITCH. ONE FLIP. DONE.

<30s

TO DISABLE SURGE PRICING GLOBALLY VIA A SINGLE KILL SWITCH

Uber. Production incident. Surge pricing off.

Never Use random() for Rollouts

Consistent user experience requires deterministic assignment.

```
// Wrong – different result every evaluation
if (Math.random() < 0.10) enableFeature();

// Right – stable per user, consistent across services
const bucket = hash(userId) % 100;
if (bucket < rolloutPercent) enableFeature();
```

Meta Gatekeeper: Zero Engineer Involvement

Auto-ramp from 0.1% to 100% overnight while you sleep.

- Gatekeeper monitors error rate, latency, and crash signals live
- Automatically advances rollout percentage when metrics stay green
- Rolls back instantly if any signal degrades past threshold
- A flag can go from 0.1% to 100% with no human in the loop
- This is progressive delivery done at planetary scale
- Build the guardrails — then let the system drive

FLAG COUNT GROWS LINEARLY. SYSTEM STATES GROW EXPONENTIALLY.

125,899,906,842,6

POSSIBLE SYSTEM STATES WITH JUST 50 FEATURE FLAGS (2 TO THE 50TH POWER)

The Combinatorial Explosion Nobody Warns You About

Atlassian Had 4,000+ Flags. On-Call Lost.

Flag debt is as dangerous as technical debt. Treat it that way.

- 4,000 active flags made incident diagnosis nearly impossible
- On-call engineers could not reason about live system state
- Flags with no owner, no expiry, and no cleanup policy compound fast
- Every flag needs an owner, a TTL, and a removal ticket
- Knight Capital's fatal flag was deprecated – just never deleted
- Your cleanup policy is part of your production safety system

Build the Control Plane. Not Just the Config.

Full deep-dive with architecture diagrams in the article.

- ✓ Control plane vs data plane – full architecture breakdown
- ✓ Kill switch evaluation order and implementation patterns
 - ✓ Deterministic hash rollouts with sticky assignment
- ✓ Auto-ramp with metric-gated progression like Meta Gatekeeper
 - ✓ Flag hygiene: TTLs, ownership, and removal workflows

Follow for more System Design deep-dives

@sairam.dev • Full article link in comments

Netflix: Flags at the Edge

Why gateway-layer evaluation changes everything

NETFLIX — TREBUCHET

Kill switches that fire before your app even wakes up

Most teams evaluate flags inside the service. Netflix evaluates them at the **API gateway** — before the request reaches application logic. The variant assignment is attached to the request context and propagated through every downstream service automatically.

The result: a kill switch doesn't just stop a feature. It stops **traffic itself**. Homepage A/B tests run on tens of millions of users simultaneously, with consistent variant assignment across the entire session — even across microservices that never talk to each other directly.

Gateway-layer evaluation = session-consistent variants

KEY INSIGHT

Where you evaluate flags determines what you can control

In-process SDK → controls feature logic | Gateway layer → controls traffic itself

Production War Story: The \$0 Bug That Cost 3 Days

What happens when two SDK versions use different hash seeds

REAL INCIDENT — FINTECH, GRADUAL CHECKOUT ROLLOUT

Same userId. Different variant. Every request.

Two SDK versions were deployed simultaneously during a gradual rollout. Both used `hash(userId) % 100` — but with different seeds. The same user evaluated into different rollout buckets on different servers. Result: alternating old and new checkout UI **on consecutive page loads**. Invisible in standard logs. Undetectable in staging.

~3%

users affected

3 days

to diagnose

1 line

to fix

0 tests

caught it

HOW IT WAS FOUND

Flag exposure tracking revealed the same userId receiving different variant assignments. Without exposure events, this would have lived forever in "flaky UX" tickets.

Every Major Tech Company Uses the Same Playbook

Arrived independently. Same architecture.

ATLASSIAN

4,000+ flags broke on-call

Mandatory 90-day TTL + auto-cleanup tickets. Engineers could no longer reason about live state.

GITHUB

Flipper – open source flag system

Percentage rollouts, actor targeting, and group targeting. Used in production for all major GitHub feature launches.

AMAZON / AWS APPCONFIG

Bake windows with auto-rollback

Config pushes include a bake time window. CloudWatch alarms monitored during rollout. Automatic rollback if error rates spike.

GOOGLE

Exposure events → ABACUS

Every flag evaluation emits a structured event. Feed directly into their experimentation platform. Causal inference without manual instrumentation.

THE PATTERN

Local evaluation. Push distribution. Kill switches first. Lifecycle enforcement by automation.

This is not four companies making the same choice. It is four companies discovering the only architecture that survives at scale.